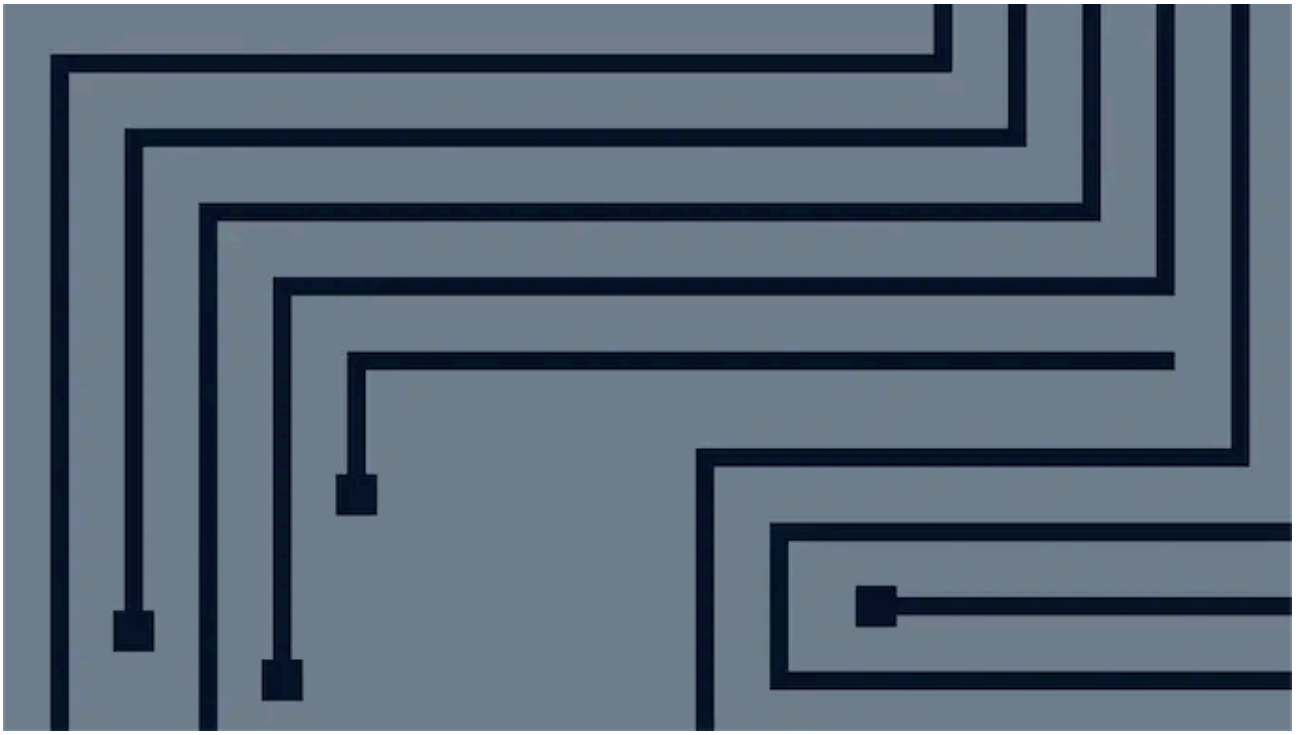
# CVE-2020-17049: Kerberos Bronze Bit Attack – Theory

netspi.com/blog/technical-blog/network-pentesting/cve-2020-17049-kerberos-bronze-bit-theory

Jake Karnes December 8, 2020



## Introduction and Background

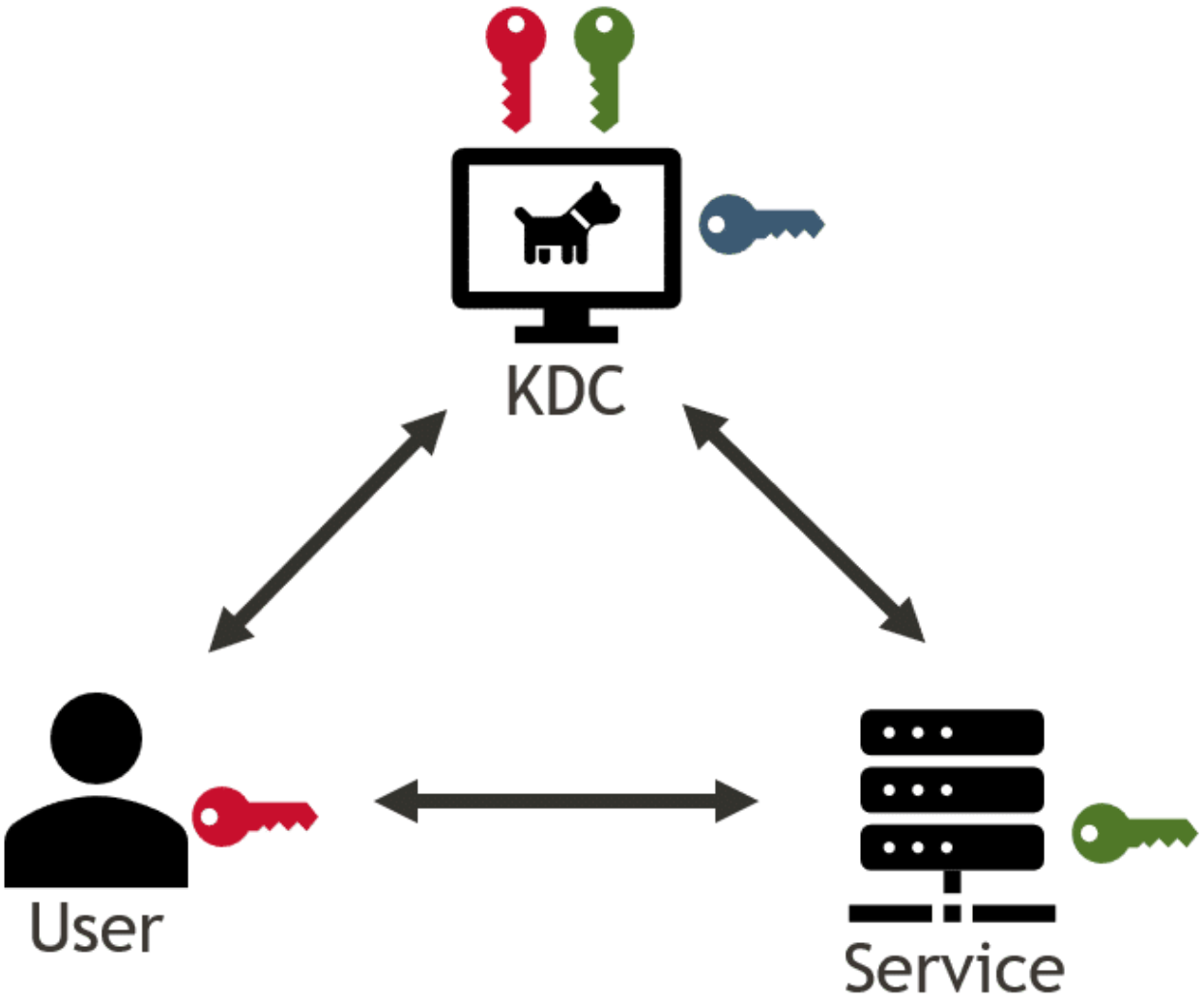
This attack expands upon the excellent research documented by Elad Shamir in "Wagging the Dog: Abusing Resource-Based Constrained Delegation to Attack Active Directory." I'll cover the key points below, but his article a great resource and primer for Kerberos and constrained delegation in AD.

If you're already familiar with the Kerberos fundamentals, feel free to skip to the Kerberos Delegation section. And if you already have a good grasp on delegation (or read the article linked above), you can jump straight into the vulnerability here. If you want to skip this background knowledge entirely and jump straight into the exploit, please see my CVE-2020-17049: Kerberos Bronze Bit Attack – Practical Exploitation post.

I'll acknowledge upfront that Kerberos is a complex protocol with many extensions, options and details. Of course, all of these are very important when discussing a security system but they cannot all be captured in a blog post. The explanations below provide a simplified view of Kerberos and Active Directory, focusing on the key points to understand the discovered weakness and its exploit.

## Kerberos 101

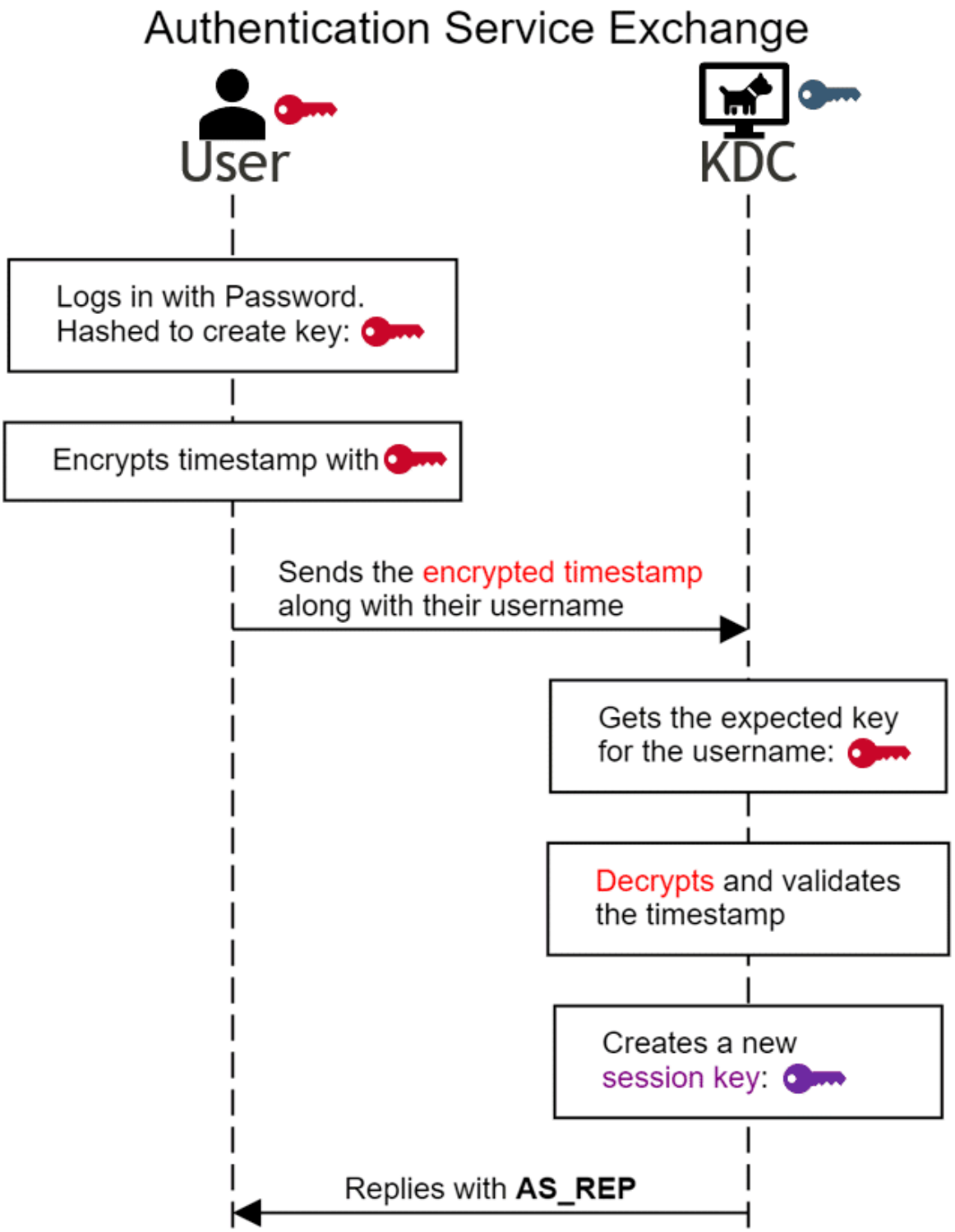
Kerberos is a protocol used in Windows Active Directory to authenticate users, servers and other resource to each other within a domain. Kerberos is based on symmetric key cryptography where each principal has a long-term secret key. This secret key is only known by the principal themselves and the Key Distribution Center (KDC). In an AD environment, the domain controller performs the role of the KDC. In the illustration below, we can see the KDC has a copy of the user's key, the service's key, and its own key.




With its knowledge of each principal's secret key, the KDC facilitates authentication of one principal to another by issuing "tickets." These tickets contain metadata, authorization information and additional secret keys which can be used with future tickets. Let's explore how these tickets are created and used by principals to authenticate to each other.

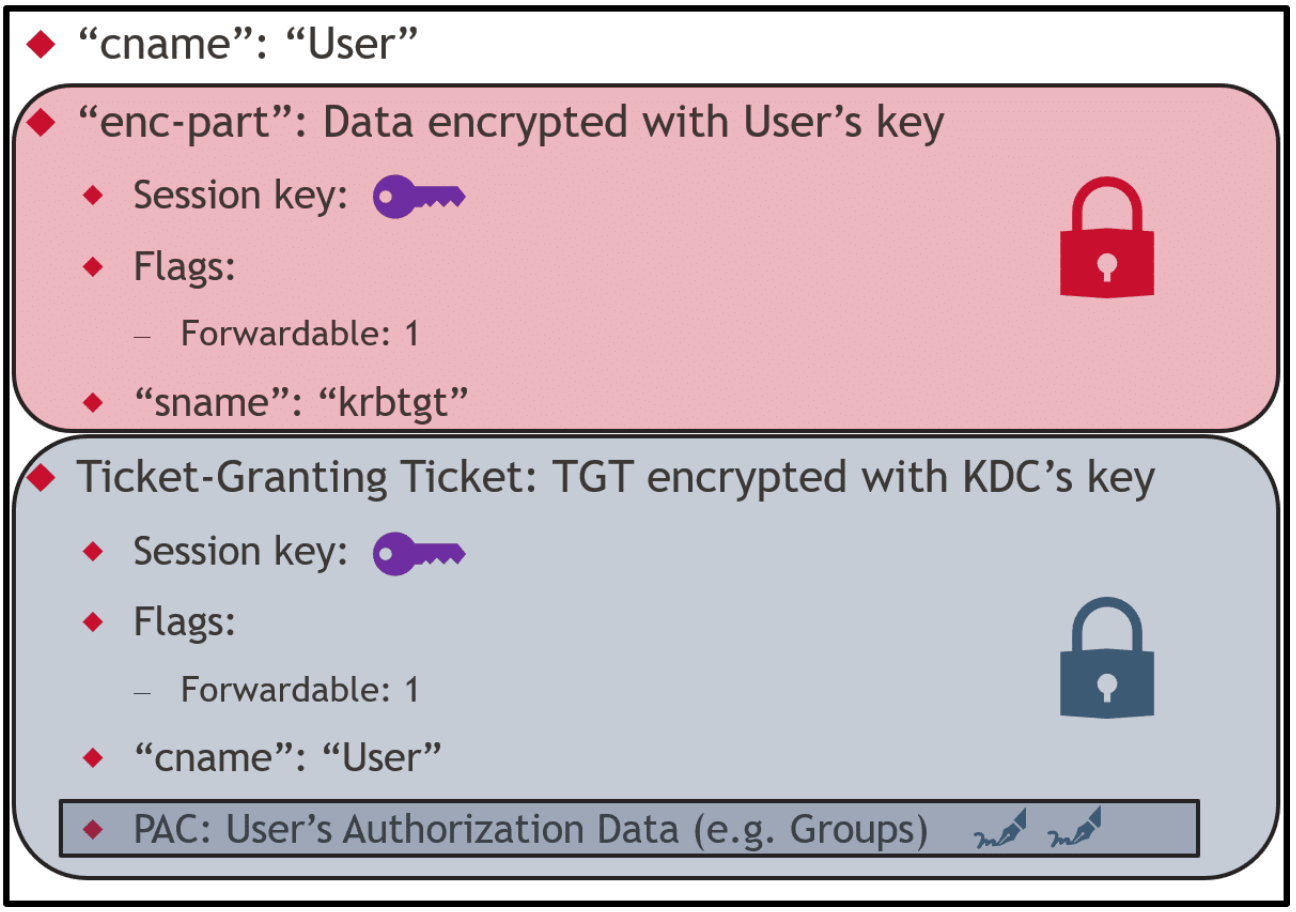
### Authentication Service Exchange

Before principals can authenticate to each other, they must authenticate themselves to the KDC. Let's take look at the process of a user authenticating themselves to the KDC through the "Authentication Service Exchange."




The user derives their secret key by hashing their password. The secret key is used to encrypt a timestamp, which is sent to the KDC along with their username. Based on the username, the KDC gets its copy of the secret key for that user. The KDC then decrypts the timestamp. If the decryption is successful, this proves that the user has access to the secret key. If the timestamp were encrypted with any other key, it wouldn't decrypt successfully with the user's key. And if the timestamp is recent, it proves that this isn't a replay attack (i.e. the user isn't sending some old, captured encrypted timestamp). If the user is using a smartcard or Windows Hello for Business instead of a password, the user

and KDC will establish key agreement via ephemeral Diffie-Hellman instead of the encrypted timestamp, but the rest of the Kerberos process will remain the same.




Once the checks pass, the KDC will create a new random logon session key, and send back an "AS_REP" reply. This AS_REP is a pretty complex data structure, so let's consider a simplified representation:

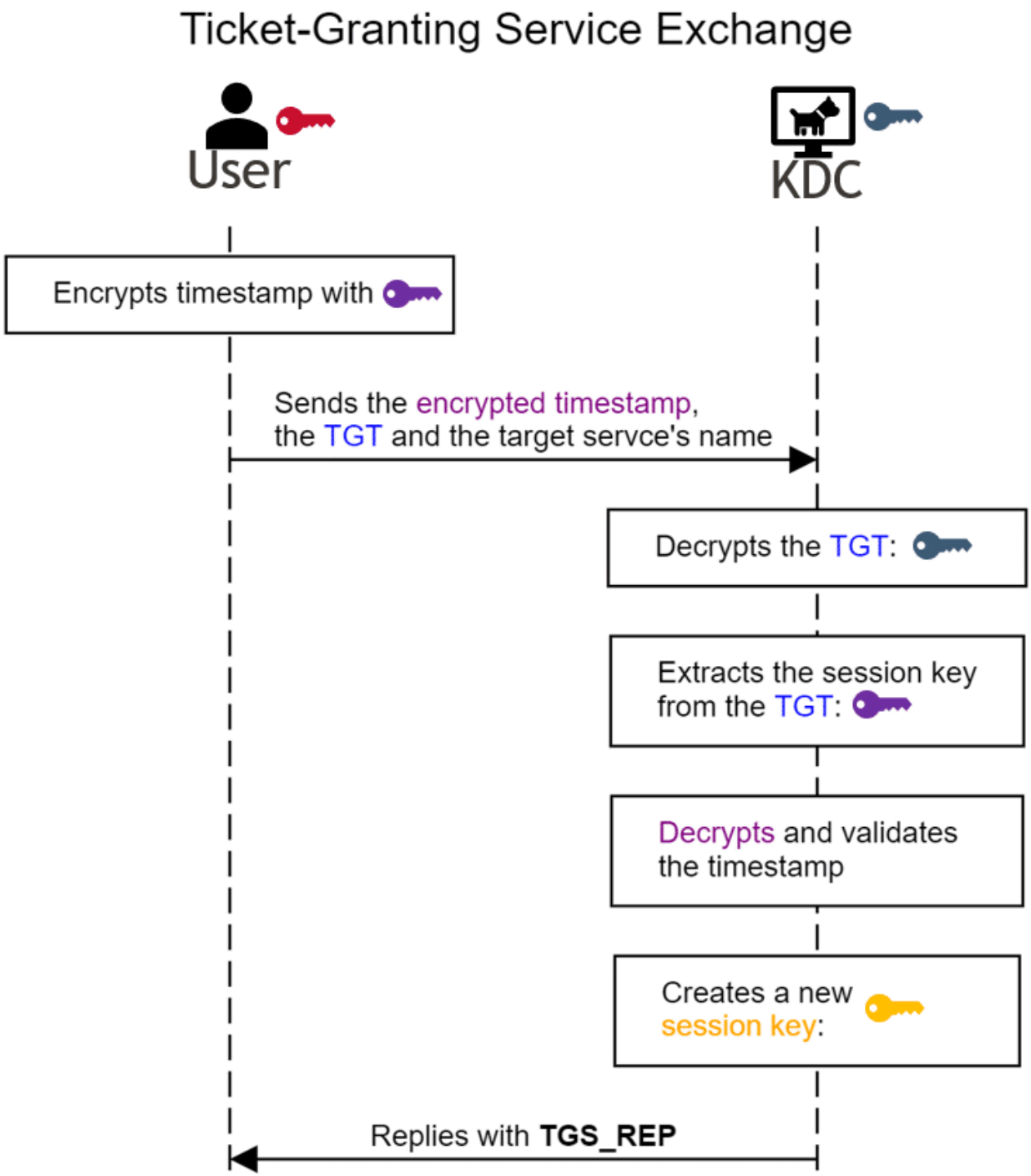
The "cname" field contains the username in plaintext. The rest of the data is split into two halves, each encrypted with a different key. The first "enc-part" section of the response is encrypted with the user's secret key. Only the user themselves can read these contents. It contains some metadata (such as "flags" and "sname" fields), and the logon session key that the KDC generated.

The last section is called the "Ticket-Granting Ticket" or TGT. This is encrypted with the KDC's secret key which means the user cannot read the contents of this section.

The user can extract the logon session key from the readable "enc-part" and the TGT from the AS_REP, and save these in their key cache for later user. The TGT can now serve as evidence that this user has already been authenticated by the KDC. I like to think of the TGT like the stamp you get on your hand when leaving an amusement park. The fact that your hand is stamped shows you were already allowed to be inside, so they'll let you back in without buying another pass.

The TGT is a very powerful authentication artifact which enables the user to prove their identity and obtain access to other services within the domain. If an attacker had access to the KDC's secret key, they could forge these tickets to impersonate any user to any other service. This is known as a Golden Ticket attack discovered by Benjamin Delpy.

### Ticket-Granting Service Exchange




After obtaining the logon session key and TGT, the user can now obtain tickets for other specific services through the Ticket-Granting Exchange:

The user begins by encrypting another timestamp, but now using the logon session key rather than their secret key. This encrypted timestamp is sent to the KDC, along with the

TGT and which service the user is requesting a ticket for. The KDC decrypts the TGT using its secret key and extracts the logon session key. The KDC then confirms the user's encrypted timestamp using the logon session key and, if the checks pass, will generate a new random service session key. The KDC sends back a "TGS_REP" reply and the exchange is complete.

Let's inspect that TGS_REP data structure as we did before:




The data structure is the same as the AS_REP, with some important differences in the values. The "enc-part" is now encrypted with the logon session key and contains the new service session key. The user can decrypt the "enc-part" using the logon session key from the key cache, extract the service session key and save it alongside the service ticket in the key cache.

The service ticket is encrypted with the service's long-term secret key and therefore can only be read by the service and KDC who have the service's secret key, not the user. You'll notice that its structure looks the same as the TGT, and that's no accident. The TGT is just a service ticket to the KDC's Ticket Granting Service. Let's take a closer look at those parts within the service ticket:

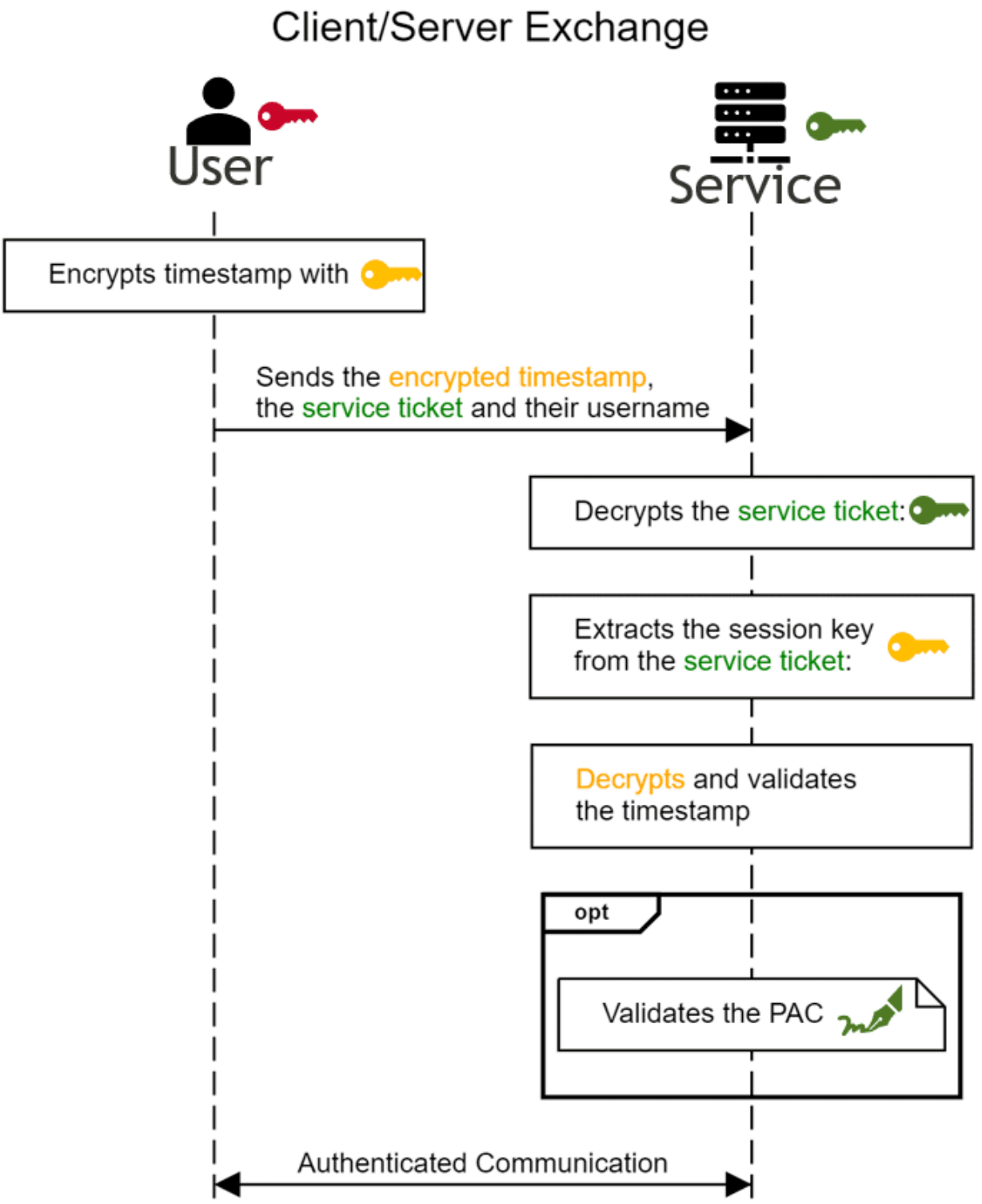
- Session key: The service session key generated by the KDC which will be shared by both the user and the service.

- Flags: A collection of binary values (only 0 or 1) which provide metadata for the ticket. The “Forwardable” flag is one of several available flags and will be discussed in detail later.
- “cname”: The client name (i.e. an identifier for the user associated with this service ticket).
- PAC: The user’s authorization data in a structure known as the privilege attribute certificate or PAC. This contains additional user metadata such as their group memberships. The PAC contains two cryptographic signatures. One signature created with the service’s key and the other created with the KDC’s key. Because of these dual signatures, the PAC is effectively tamper-proof.

It’s also worth noting that the “sname” field (which identifies the service’s name) is not in the encrypted service ticket. This allows for interesting sname substitution attacks as first described by Alberto Solino.

### Client/Server Exchange

With a service ticket and a service session key, the user is finally ready to authenticate to the service.




I think we can see the familiar pattern here. A timestamp is encrypted with the service session key, and sent from the user to the service along with the service ticket. The service decrypts the ticket with its long-term key. This proves the ticket came from the KDC, since only the KDC and the service itself should have the long-term key to create such a service ticket. The service session key is extracted from the service ticket and used to validate the timestamp.

The service will then use its long-term key to check the signature of the PAC. This provides the user's authorization upfront, so the service doesn't need to fetch it from the KDC. Optionally, the service may choose to send the PAC to the KDC, so that the KDC may

validate the second signature against the KDC's key. If performed, this optional check verifies that the PAC has not been altered from when it was first created by the KDC and helps prevent attempts to forge the PAC and escalate privileges.

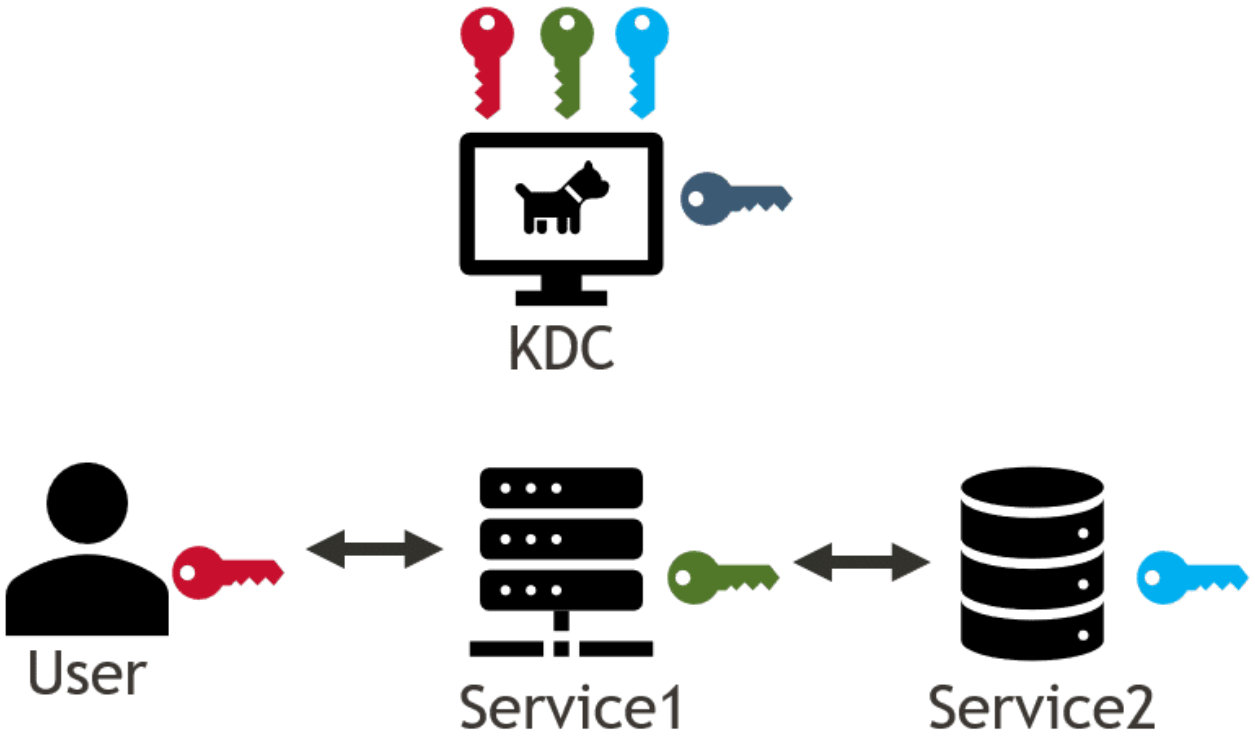
Once these checks are performed and passed, the user has successfully authenticated to the service, the service is aware of the user's authorization, and the user may proceed with any requests.

Like the TGT, if an attacker can forge a valid service ticket to a particular service, then it can impersonate any user and authenticate to that service. This is known as a "Silver Ticket" attack and requires knowledge of the service's secret key. A popular technique for obtaining a service's secret key is "Kerberoasting" discovered by Tim Medin. However, this attack is thwarted if the service performs the optional step of sending the PAC to the KDC. The KDC will observe that its signature for the PAC is incorrect (because the attacker doesn't have the KDC's secret key), and it will report that error back to the service which will reject the service ticket.

## Kerberos Delegation

Now that we understand the fundamentals of how Kerberos authenticates a user to a service, consider the following: what if that service wants to send requests to another service on the user's behalf? A common scenario would be when a user authenticates to a web application, and that web application needs to access a database under the user's authority. The database controls whether a given user is allowed to access a particular record. When the web application attempts to access a database record, it must carry the user's authorization in the request to allow the database to determine if the request should be allowed or denied. This is sometimes referred to as the "double-hop" scenario.

For our examples going forward, "Service1" will be the service which the user is directly authenticated to, and "Service2" will be the additional service which needs to be accessed under the user's authority.




This problem is addressed through “Kerberos delegation” which allows Service1 to impersonate the user and interact with Service2 as if the requests came directly from the user. There are several types of Kerberos delegation supported in Active Directory which will be discussed in detail below:

- Unconstrained Delegation
- Constrained Delegation
  - Without Protocol Transition
  - With Protocol Transition
- Resource-Based Constrained Delegation

The following screenshot captures where most of the delegation configuration lives in Active Directory:

### Unconstrained Delegation

Although the vulnerability doesn’t impact this configuration, unconstrained delegation should be avoided. Let’s consider that Service1 has been configured for unconstrained delegation with the “Trust this computer for delegation to any service (Kerberos only)” setting above. When a user obtains a service ticket to Service1, the KDC will embed the user’s TGT into the service ticket. When the service ticket is passed to Service1, the service can extract the TGT. With access to the user’s TGT, Service1 can perform the Ticket-Granting Service Exchange, and obtain service tickets as the user to any other service. This allows complete impersonation of the user.

Issues arising from unconstrained delegation are well-known and have been discussed thoroughly by Will Schroeder here and Sean Metcalf here. Since this is no longer recommended, may not even be supported, and not directly related to the vulnerability, I'll refer you to those great resources for a deeper dive.

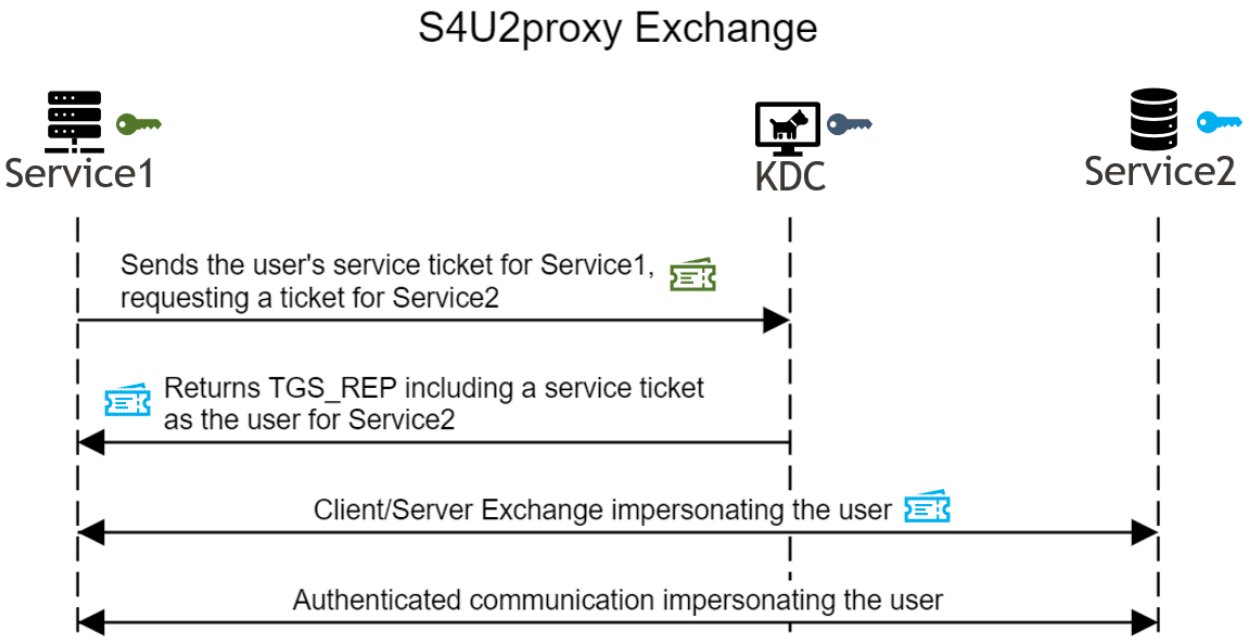
## Constrained Delegation

Due to the potential issues with unconstrained delegation, Microsoft introduced the concept of "constrained delegation" in its Windows Server 2003 release, and published [MS-SFU]: Service for User and Constrained Delegation Protocol as a public specification and Kerberos extension in 2007. Unlike unconstrained delegation, the extension would allow for services' delegation targets to be predefined. Using our running example, it could be configured that Service1 is only allowed to delegate to Service2, instead of every service in the domain. When the user obtains a service ticket for Service1, the KDC would no longer embed the user's TGT into the service ticket.

### Constrained Delegation without Protocol Transition

Let's consider that a user has authenticated to Service1 through Kerberos, passing their service ticket, and now Service1 needs to access Service2 under the user's authority. Because the user's service ticket is specific to Service1, it would be rejected if passed directly to Service2. Without the user's TGT, Service1 cannot obtain a service ticket to Service2 from the KDC. So how does the Service1 authenticate to Service2 as the user?

The MS-SFU specification solves this problem through the "Service for User to Proxy" (S4U2proxy) protocol. The S4U2proxy protocol allows Service1 to obtain a service ticket to Service2 as the user, by sending the KDC the service ticket it received from the user, along with Service1's own TGT. The KDC will reply with a "TGS_REP" including a service ticket which valid for Service2, authorized as the user. Once Service1 has a service ticket as the user to Service2, it can proceed with the same Client/Server exchange as before. After that exchange is complete, Service2 will process the requests from Service1 as if they came from the user themselves.

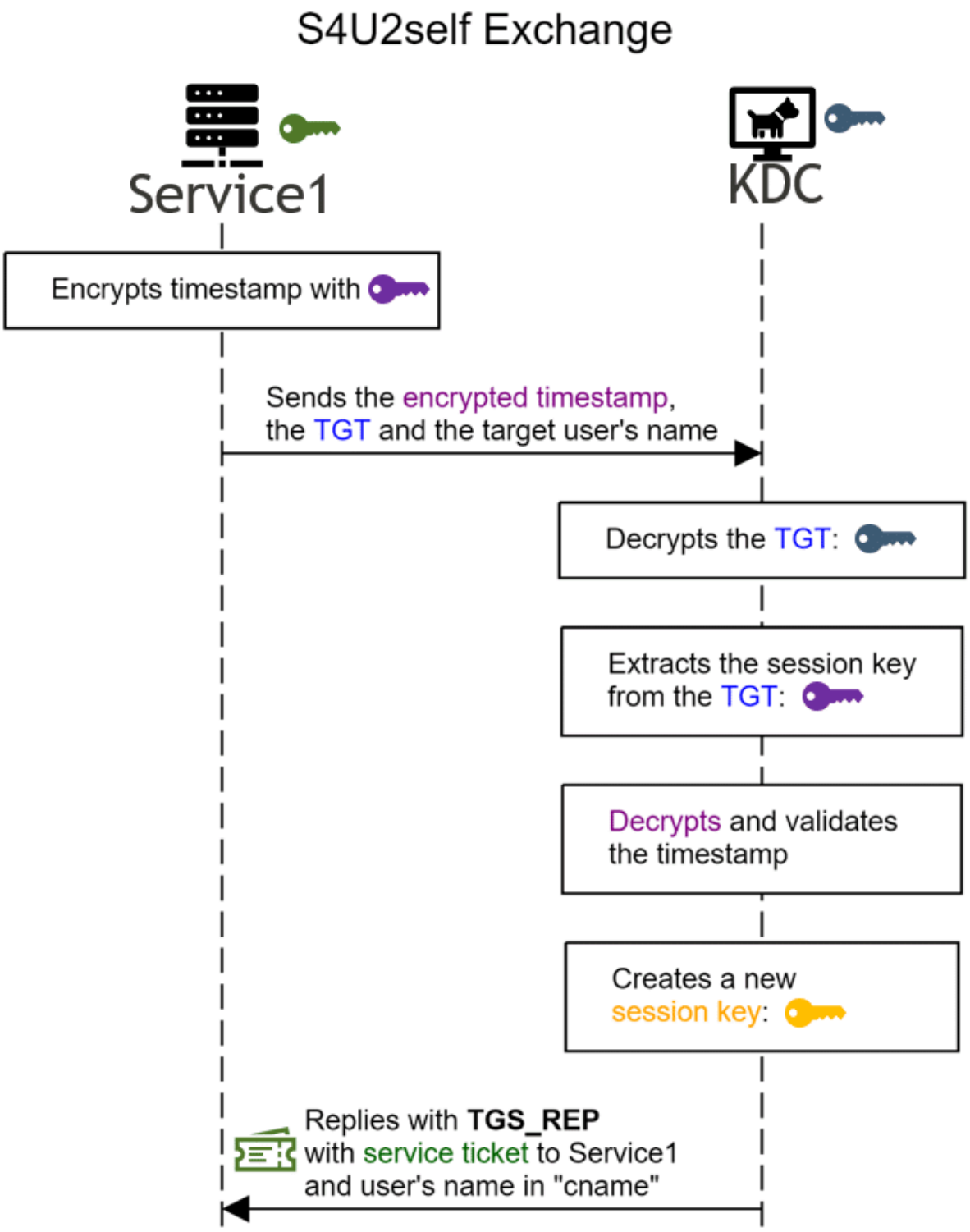



The S4U2proxy protocol allows for constrained delegation “without protocol transition” because the Kerberos protocol is used for every step. All principals are authenticated to each other through Kerberos and its ticket exchanges.

#### Constrained Delegation with Protocol Transition

What if the user authenticated to Service1 by a protocol other than Kerberos? For example, Service1 may have authenticated the user through NTLM v2 Authentication. If Service1 then needed to delegate that authentication to Service2, it would be unable to do so. The user never presented a service ticket to Service1, so Service1 cannot pass that service ticket to the KDC in the S4U2proxy protocol. In this case, Service1 would need to perform a “protocol transition” using the second protocol provided in the MS-SFU specification: the “Service for User to Self” (S4U2self) protocol.

The S4U2self protocol allows a service to obtain a service ticket to itself on behalf of any user. The protocol is a modification of the Ticket-Granting Exchange. Service1 presents its own TGT and timestamp encrypted with its logon session key to the KDC and specifies which user it would like a ticket for. The KDC processes the request, performing the same validations as before, and returns a service ticket for Service1 which identifies the specified user as the client in the “cname” field.




Now that Service1 has obtained a service ticket to itself, it can present this ticket as evidence to in the S4U2proxy exchange and obtain a service ticket to Service2 on the user's behalf.

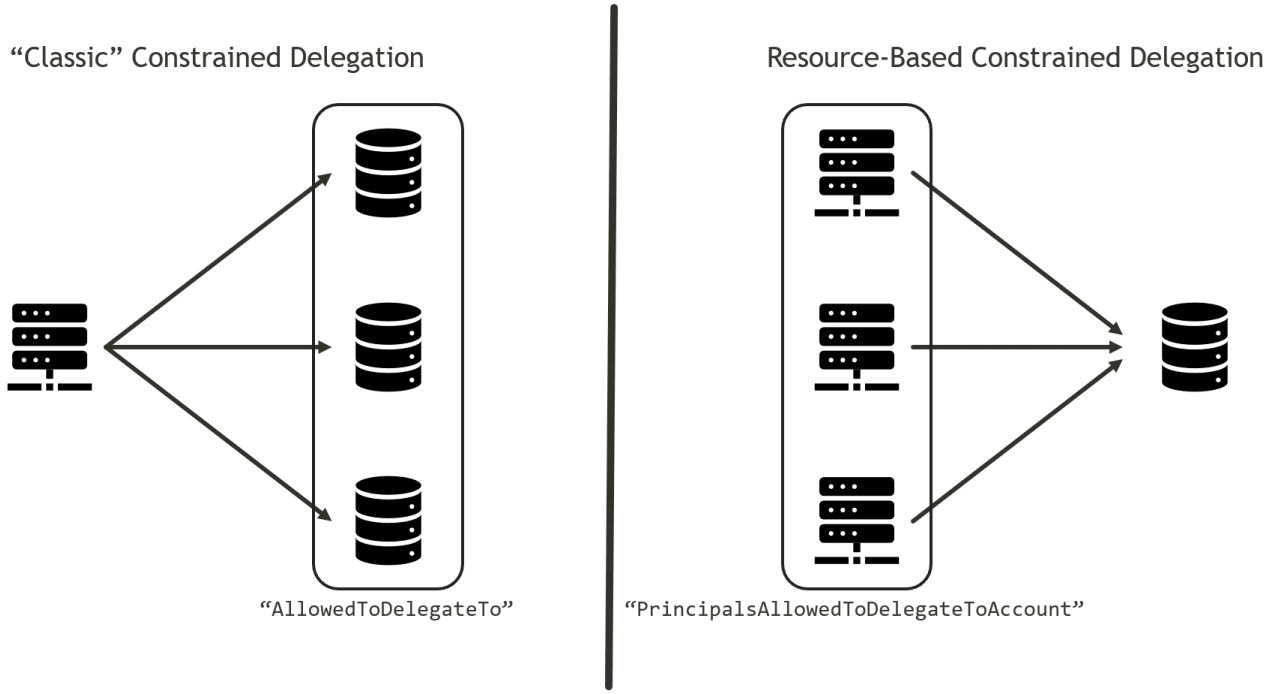
It's important to note that a service can obtain a service ticket to itself on behalf of any user through the S4U2self protocol. The service is not required to present any evidence that the user has actually authenticated to the service.

## Resource-Based Constrained Delegation (RBCD)

When Microsoft first introduced the concept of Kerberos constrained delegation, it was only possible to define a list of services which a specified service could delegate to. For example, a domain admin could specify that ServiceA is allowed to delegate Kerberos authentication to ServiceB, ServiceC and ServiceD. The list of delegation targets would be configured in the “AllowedToDelegateTo” property of ServiceA in Active Directory.

Resource-based Constrained Delegation flips this model. Introduced in Windows Server 2012, RBCD lets a service define which other services it accepts delegated Kerberos authentication from. For example, a privileged user (not necessarily a domain admin) could specify that ServiceW accepts delegated Kerberos authentication from ServiceX, ServiceY and ServiceZ. This list would configured in the “PrincipalsAllowedToDelegateToAccount” property of ServiceW in Active Directory.

After the introduction of RBCD, the existing constrained delegation became unofficially known as “classic” constrained delegation.




## Protections Within Constrained Delegation

While the scope is more narrow than unconstrained delegation, constrained delegation still allows for the impersonation of users and therefore needs strong security controls to limit its potential impact if abused. Microsoft provides a variety of configuration options to balance security and functionality.

### Allowing and Disallowing Protocol Transition

When using “classic” constrained delegation, the sysadmin must decide if the service is allowed to perform the “protocol transition” described previously. If so, the service will be permitted to impersonate users who haven’t already been authenticated through Kerberos.

This trusts that the service will only impersonate users who it has authenticated through another mechanism.

If the service is trusted for constrained delegation with protocol transition, then the “TrustedToAuthForDelegation” property in Active Directory is enabled. This corresponds to the “Trust this computer for delegation to specified services only – Use any authentication protocol” option in the AD GUI.

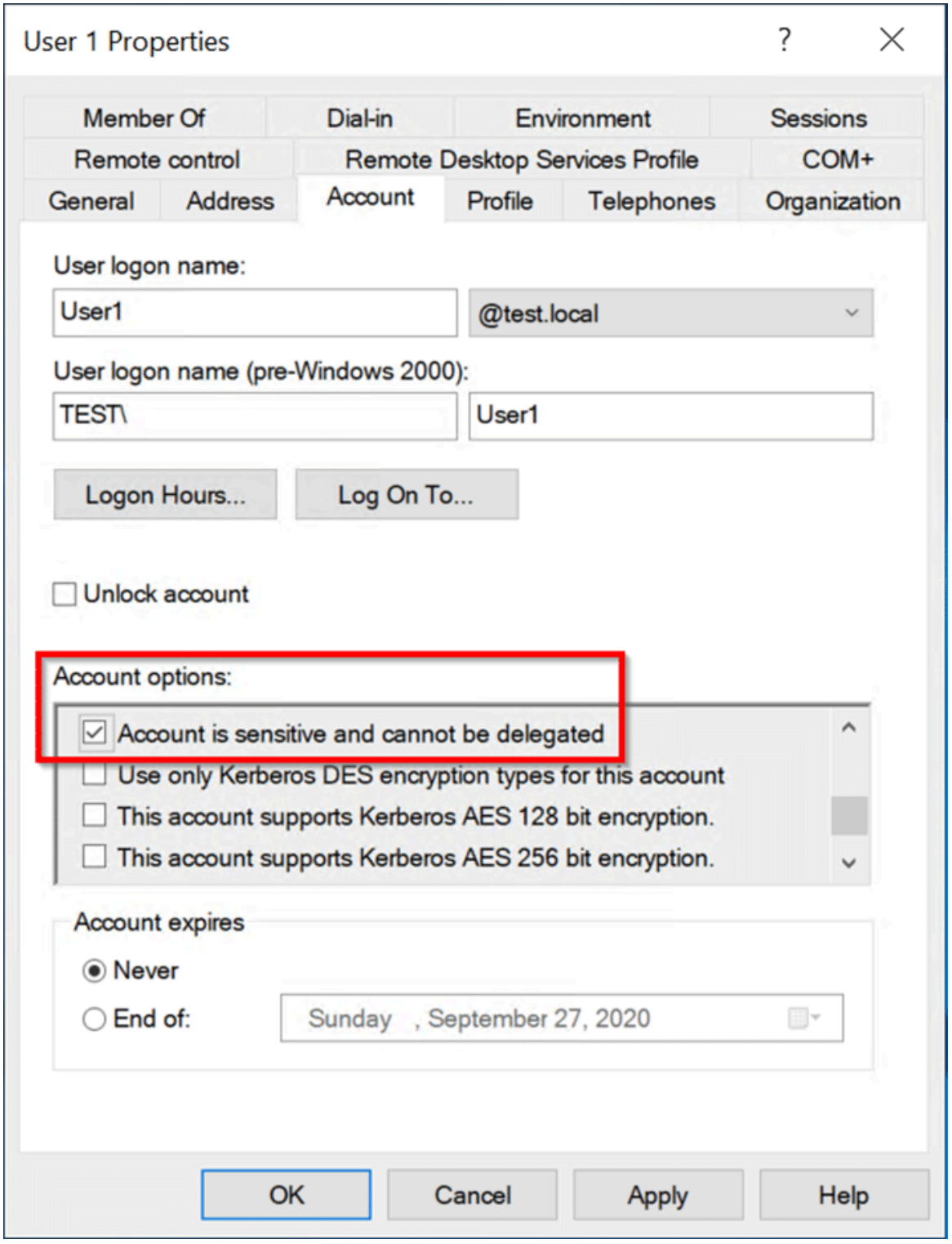
If the service is trusted for constrained delegation without protocol transition, then the “TrustedToAuthForDelegation” property in Active Directory is not enabled but the AllowedToDelegateTo list is still populated with delegation targets.  This corresponds to the “Trust this computer for delegation to specified services only – Use Kerberos only” option in the AD GUI. This restriction is enforced by the S4U2self protocol. While any service can perform the S4U2self exchange to obtain a service ticket to itself as any user, if protocol transition is not allowed then the resulting service ticket will have its Forwardable flag set to 0. When a service ticket with a Forwardable flag of 0 is passed in the subsequent S4U2proxy exchange, the exchange will fail.

Effectively, “TrustedToAuthForDelegation” allows a service to use both the S4U2self and the S4U2proxy protocols successfully. If a service is not “TrustedToAuthForDelegation” but has a non-empty AllowedToDelegateTo list, then the service may use S4U2self successfully, but the service can only pass forwardable tickets to S4U2proxy. The service should only obtain forwardable tickets from users that have already authenticated through Kerberos with the KDC directly, preventing impersonation.

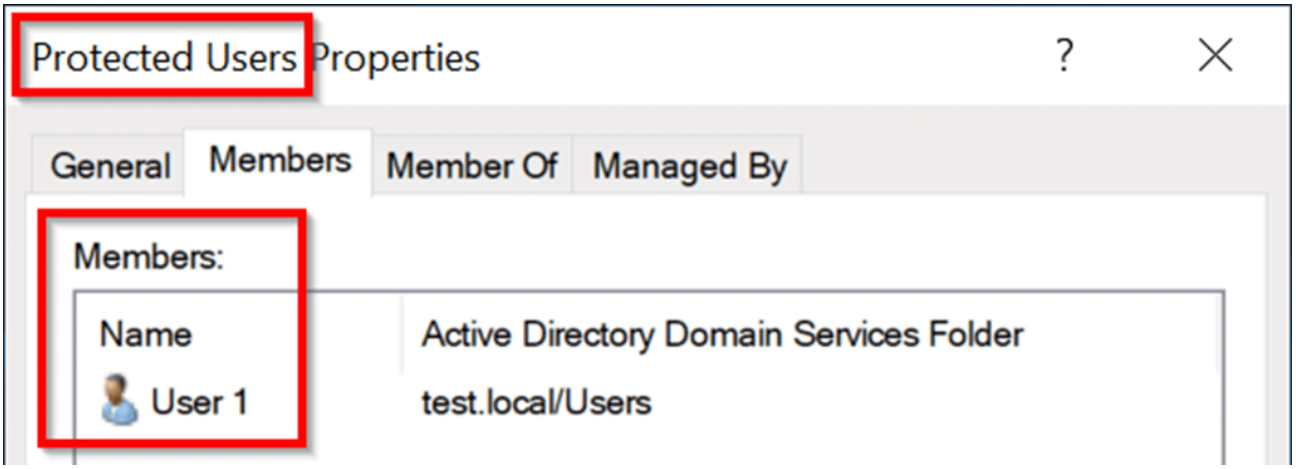
#### Protected Users and Sensitive Accounts

Services aren’t the only principals with security restrictions applied. Users and other AD accounts can be configured to disallow delegation of their authentication. This means that even if a service is allowed to perform delegation (of any kind), the service cannot delegate and impersonate the user. There are two ways of protecting an account from delegation.

First, a sysadmin can enable the “Account is sensitive and cannot be delegated” setting in Active Directory for the account. The screenshot below shows how this is displayed in the AD GUI.




The second option is to add the account to the “Protected Users” security group in AD. Any members of this group may not have their authentication delegated, and there are other protections as well.




Like the “Use Kerberos only” restriction discussed in the previous section, this protection is enforced by ensuring that all tickets issued for these users are not forwardable. When a ticket is created for a user with the “account is sensitive and cannot be delegated” setting, or if the user is a member of the “Protected Users” group, then the Forwardable flag value will always be 0. A service can still obtain a ticket for these users to itself through the S4U2self protocol, but since the ticket is not forwardable, it cannot be successfully used in the S4U2proxy protocol to obtain a ticket for another service.

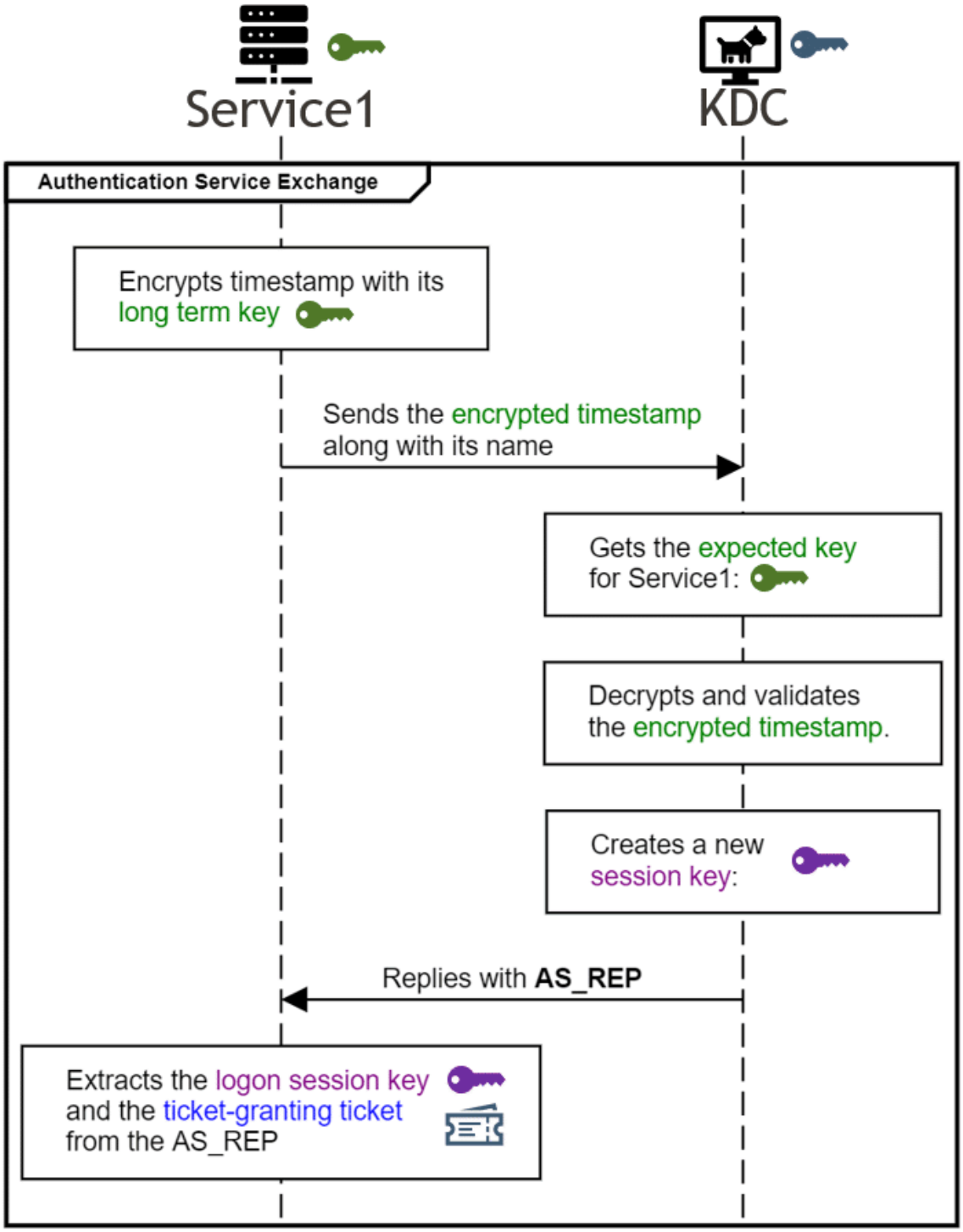
#### Enforcing Constrained Delegation Lists

Of course, the KDC must also enforce the constrained delegation lists discussed previously. This is only applicable during the S4U2proxy protocol. Let’s return to our example of Service1 attempting to delegate authentication to Service2. During the S4U2proxy protocol, the KDC will confirm that Service2 is in Service1’s “AllowedToDelegateTo” list (allowing “classic” constrained delegation) **or** that Service1 is in Service2’s “PrincipalsAllowedToDelegateToAccount” list (allowing resource-based constrained delegation). If neither of those conditions are met, then the exchange fails. This prevents a service from performing constrained delegation to another, unless one of them have been explicitly configured for that.

## The Big Picture

Now that we’ve covered the fundamentals of Kerberos, constrained delegation, and its protections, let’s bring all of this together. Let’s consider that Service1 wants to authenticate to Service2 as another user, but the user has not already authenticated to Service1 through Kerberos.

Service1 starts with the standard Authentication Service exchange. Like before, it encrypts a timestamp with its long-term secret key to prove its identity. The KDC validates the timestamp, returns a logon session key and a ticket-grant ticket (TGT) in an AS_REP response.

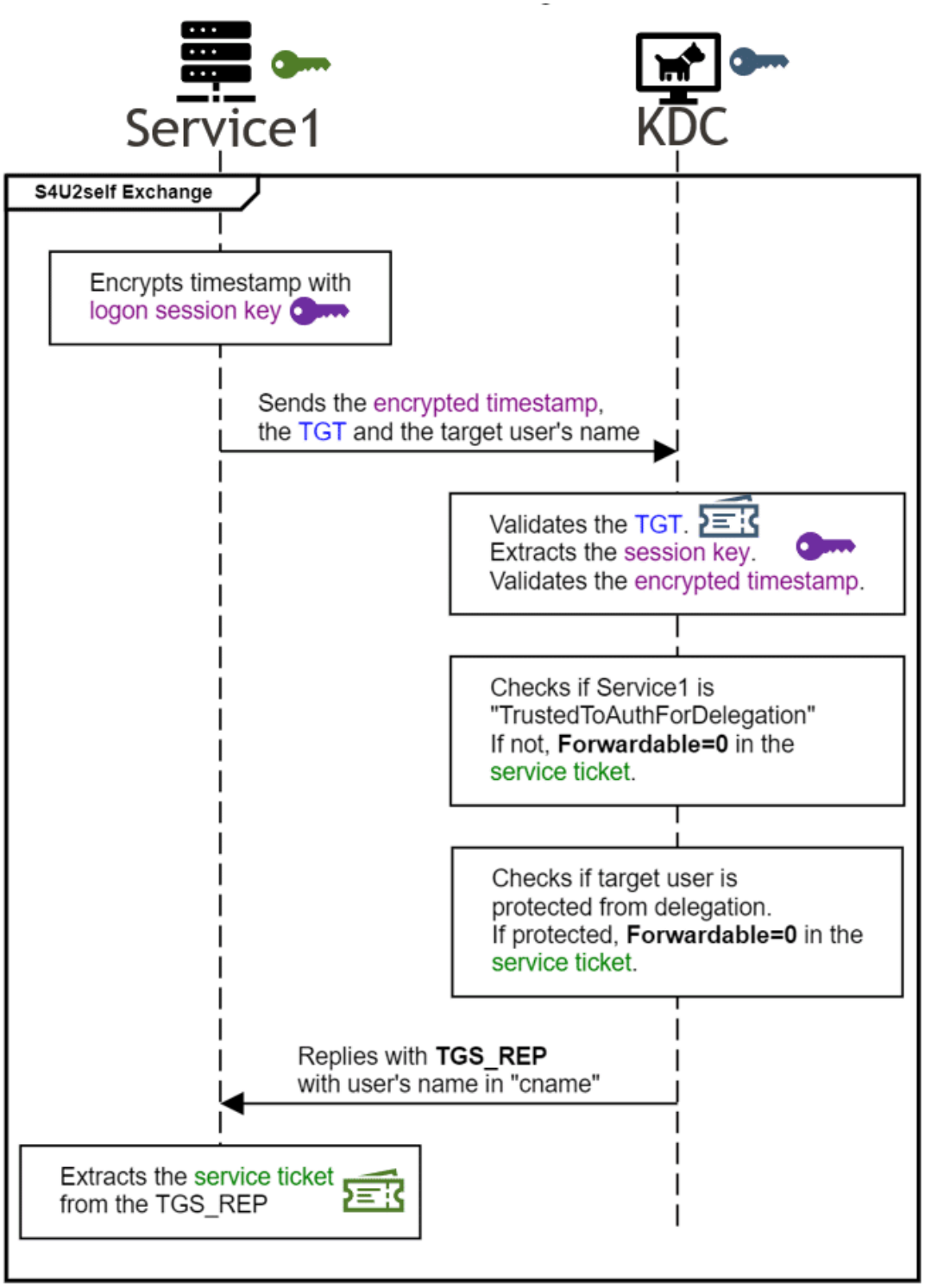



Service1 extracts the logon session key and TGT from the AS_REP, then continues to the S4U2self exchange. Service1 encrypts another timestamp with the logon session key. Service1 sends the encrypted timestamp, its TGT and the target user's name to the KDC, requesting a ticket to itself through the S4U2self protocol. The KDC will validate the incoming TGT and timestamp. If this passes, the KDC prepares a service ticket for the specified user to Service1. Initially, the service ticket's forwardable flag is set (i.e. Forwardable=1).

The KDC will check if Service1 has the "TrustedToAuthForDelegation" property set. If not, the service's ticket forwardable flag is set to 0. The KDC will also check if the target user is

protected from delegation. If the user is a member of the “Protected Users” group or configured with the “Account is sensitive and cannot be delegated” setting, then the forwardable flag in the service ticket is set to 0.

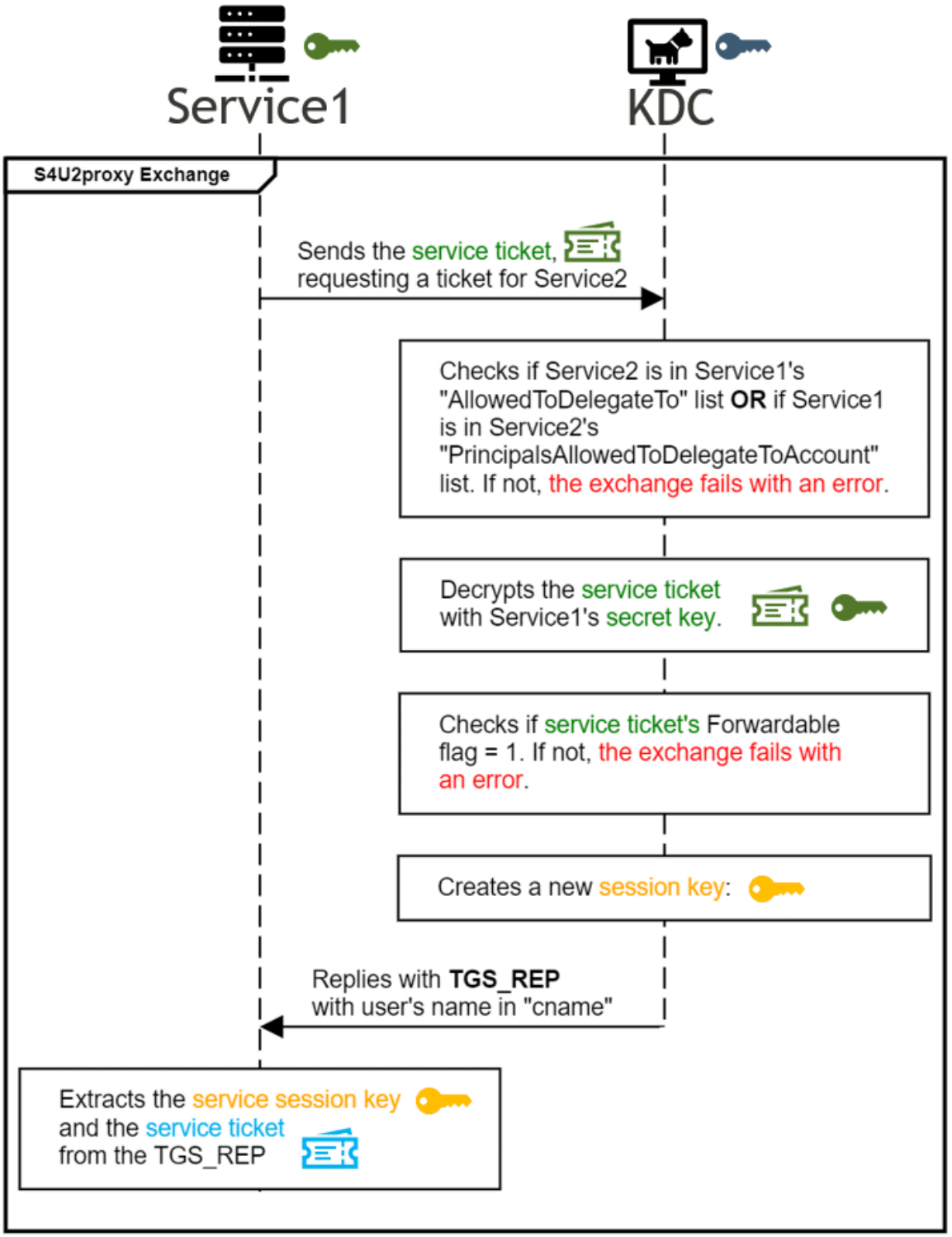
The KDC returns the new service ticket to Service1. Service1 now has a valid ticket as the user to itself, which may or may not be forwardable.




Service1 is now ready for the final step: the S4U2proxy Exchange. Service1 presents the service ticket it received back to the KDC as proof of the user’s authentication to Service1, and Service1 requests a new service ticket to Service2 as the user. The KDC begins by

confirming that there is a delegation trust relationship between Service1 and Service2. It checks if Service2 is in Service1's "AllowedToDelegateTo" list. If so, a classic constrained delegation trust relationship is confirmed. If not, the KDC checks if Service1 is in Service2's "PrincipalsAllowedToDelegateToAccount" list. If so, a resource-based constrained delegation trust relationship exists. If both checks fail, the exchange fails with an error.

After confirming that Service1 is allowed to delegate authentication to Service2, the KDC decrypts the received service ticket using Service1's long-term secret key and checks the Forwardable flag. If the service ticket's Forwardable flag is set to 0, the KDC will perform additional checks and exchange will fail with an error. If the Forwardable flag is set to 1, the KDC will return a new service ticket which is valid for Service2 as the target user.




### Spot the Problem

There you go! You now have all the information necessary to spot what it wrong in this authentication protocol. If you'd like, take a second to review the information covered so far and see if you can find the vulnerability. When you're ready, continue down to the next section.

## Flipping Bits for Fun and for Profit

Let's take a closer at the TGS_REP data structure returned by the KDC after the S4U2self exchange.  Consider the scenario where Service1 is not "TrustedToAuthForDelegation" or the specified user is protected from delegation (because it is a member of the "Protected Users" group or it is configured with the "Account is sensitive and cannot be delegated" setting). Here's how that TGS_REP could look:




Because of the protections in place, the Forwardable flag is not set (i.e. its value is 0). This means that the service ticket would be rejected if used as proof in the S4U2proxy exchange.

Look closely at where the Forwardable flag is located in the response. The service ticket's Forwardable flag is encrypted with Service1's long-term. The Forwardable flag is not in the signed PAC. Service1 is free to decrypt, set the Forwardable flag's value to 1, and re-encrypt the service ticket. Because it's not in the signed PAC, the KDC is unable to detect that the value has been tampered with.



Voila! We have converted a non-forwardable ticket into a forwardable ticket. This forwardable service ticket can be provided as proof in the S4U2proxy exchange, allowing us to delegate authentication to Service2, as any user of our choice.

### Protections Bypassed

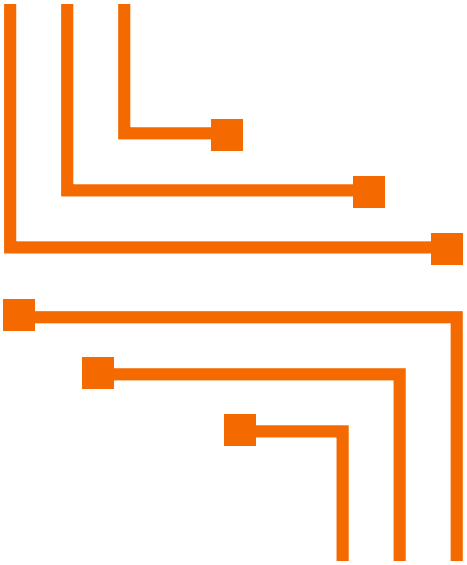
Recall the protections discussed earlier. By flipping the forwardable bit, we're bypassing two of the three protections:

1. We've bypassed the protection for TrustedToAuthForDelegation and the "Trust this computer for delegation to specified services only – Use Kerberos only" configuration. This protection is enforced by ensuring that any service ticket received in the S4U2self exchange is non-forwardable, unless the requesting service is TrustedToAuthForDelegation. By setting the forwardable flag ourselves, we've effectively removed this distinction and enabled the service to perform the protocol transition, as if the service were configured with the "Trust this computer for delegation to specified services only – Use any authentication protocol" option.
2. We've also bypassed the protection for accounts which do not allow delegation. Again, this is enforced by ensuring that any service ticket received in the S4U2self exchange on behalf of a protected account is non-forwardable. By converting this to a forwardable service ticket, the service can now delegate the account's authentication as if there was no such protection.

## Microsoft's Patch

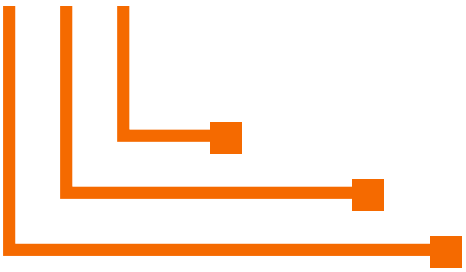
Microsoft has released multiple patches throughout November and December 2020 to fix this vulnerability. The various patches can be found in the MSRC Security Update Guide for CVE-2020-17049 and their rollout plan and advice can be found here.

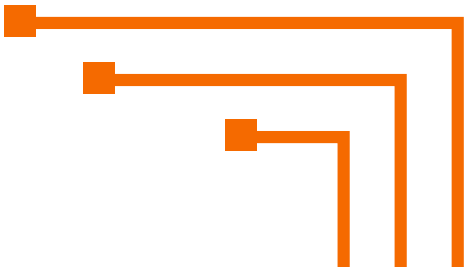
Because the "Service for User and Constrained Delegation Protocol" (MS-SFU) is an open specification, we can get a good idea of how the fix was implemented. MS-SFU revision 19.0 published on November 23, 2020 now contains the underlined addition to section 3.2.5.2.2:



> Service 1's KDC verifies both server ([MS-PAC] section 2.8.1) and KDC ([MS-PAC] section 2.8.2) signatures of the PAC. Because Service 1's KDC is ingesting a service ticket rather than a TGT, it SHOULD also ensure the integrity of the service ticket by verifying the ticket  signature ([MS-PAC] section 2.8.3). If Service 2 is in another domain, then its KDC verifies only the KDC signature of the PAC.

The specification now references a "ticket signature" which should be verified by the KDC along with the existing PAC. Let's look at that referenced section of the MS-PAC specification. We can see that revision 20.0 was published on the same day and added Section 2.8.3. I think the following snippet of that section best captures the core change:





> The ticket signature is used to detect tampering of tickets by parties other than the KDC. The ticket signature SHOULD be included in tickets that are not encrypted to the krbtgt account (including the change password service) or to a trust account. The KDC signature is a keyed hash [RFC4757] of the ticket being issued less the PAC itself.

The key takeaway for us is that the PAC now has an additional field which holds the “ticket signature.” When the service ticket is produced during the S4U2self exchange, the KDC signs the ticket contents with its secret key and inserts the signature into the PAC. As discussed previously, the PAC itself is also doubly signed with the KDC’s secret key and the service’s key. Later when the KDC receives the service ticket during the S4U2proxy exchange, the KDC can validate all three signatures to confirm that the PAC and the service ticket have not been modified. If the service ticket is modified (for example, if the the forwardable bit has changed), the KDC will detect the change and reject the request with an error such as “KRB_AP_ERR_MODIFIED(Message stream modified).”

## Final Thoughts

Thank you so much for joining me on this journey through Hades to get to know Kerberos. Armed with this background knowledge, I hope you continue onto my CVE-2020-17049: Kerberos Bronze Bit Attack – Practical Exploitation post. There I discuss how the attack is implemented and how it could be used. I hope to see you there!

## Explore More Blog Posts



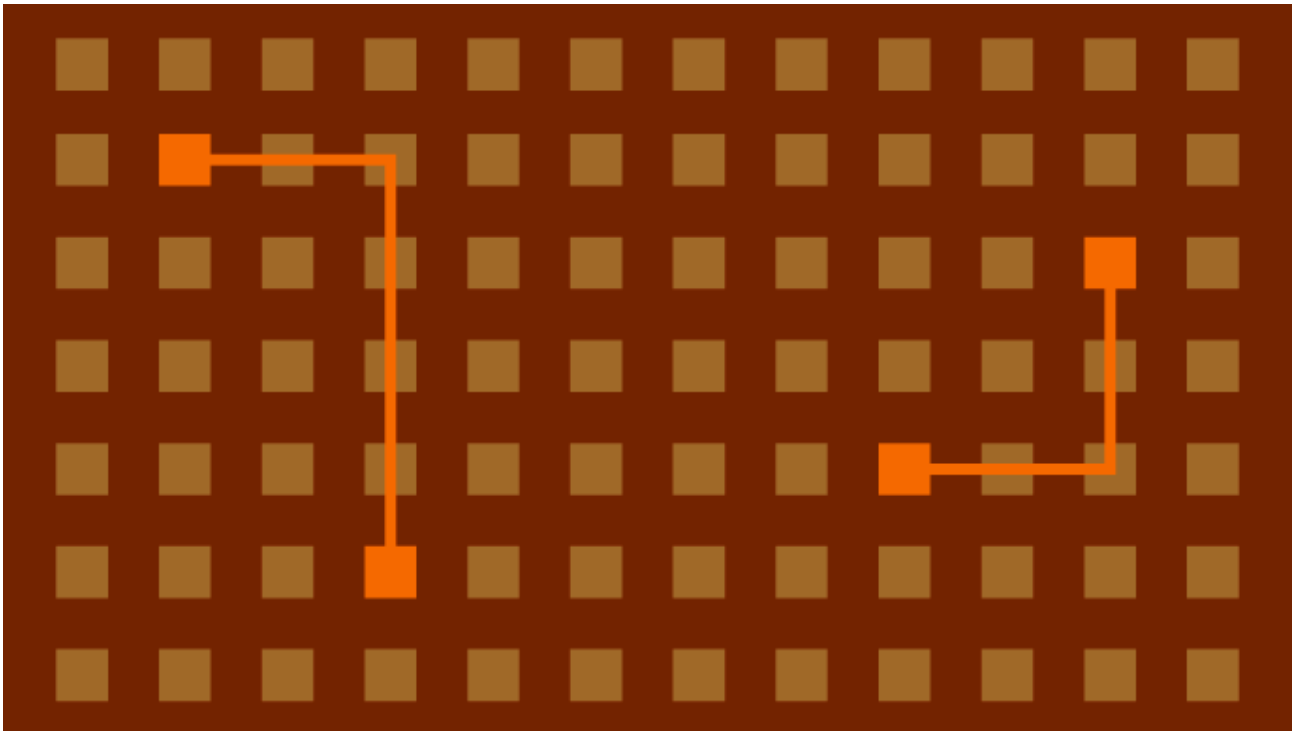
Cloud Pentesting

### Administrating the Administrator, An OCI Identity Domains Permission Bypass

October 13, 2026

Discover how a permission bypass in Oracle Cloud Infrastructure (OCI) identity domains allowed lower-level administrator roles to create API keys for Identity Domain Administrators, leading to an unauthenticated privilege escalation path.

Learn More



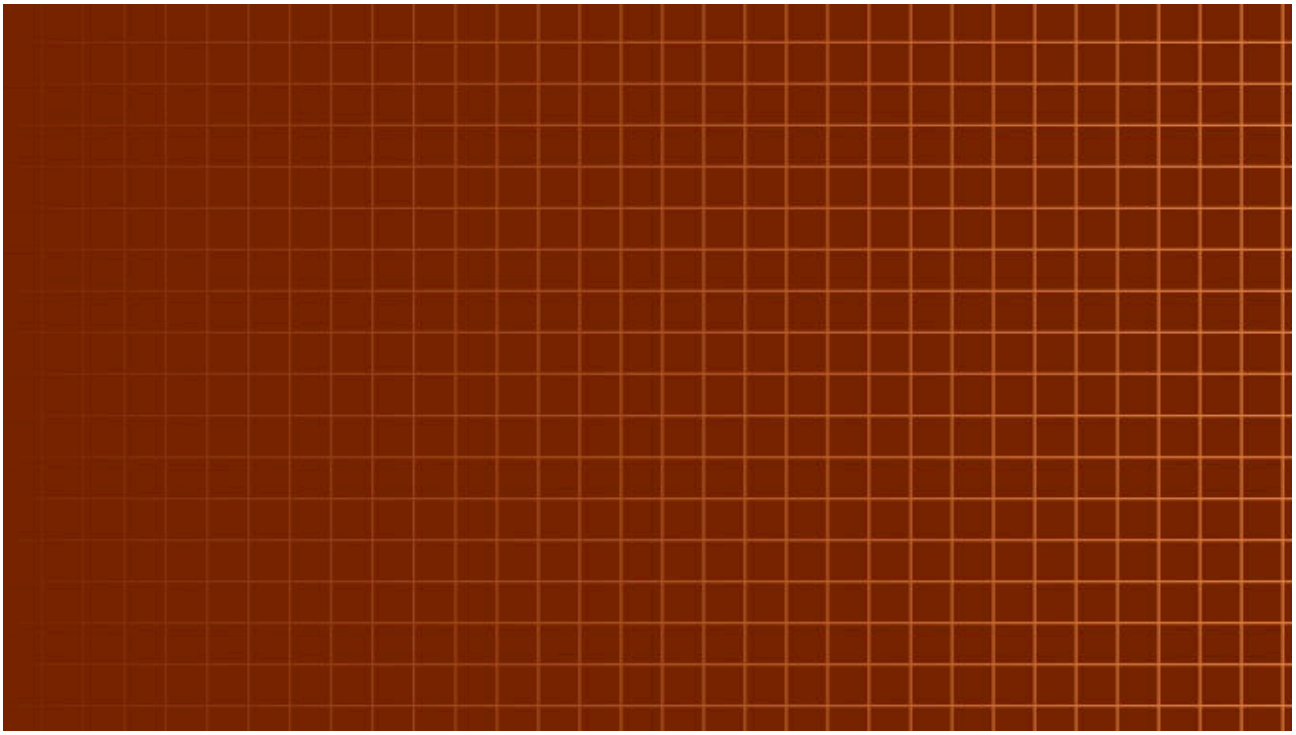
Web Application Pentesting

## CVE-2026-78902: XSS to RCE in pfSense with one DNS request

September 22, 2026

Discover how a single DNS request can chain a stored Cross-Site Scripting (XSS) vulnerability into Remote Code Execution (RCE) on pfSense firewalls. Read the full technical breakdown of CVE-2026-78902 affecting the popular pfBlockerNG package.

Learn More



Critical Vulnerability

## “StyleSmuggler” – Adobe Commerce, Adobe Commerce B2B, and Magento RCE (CVE-2026-75650): Overview and Takeaways

September 8, 2026

Adobe disclosed CVE-2026-7565, dubbed “StyleSmuggler,” on September 7th, 2026. The vulnerability, which received a maximal initial CVSS score of 10.0, affects Adobe Commerce, Adobe Commerce B2B, and Magento Open Source. The flaw allows an unauthenticated network-based attacker to execute arbitrary code by injecting PHP through Magento's email template engine.

Learn More